

Modelo y Arquitectura de Base de Datos

NoSQL (Firestore) - iBichos

El proyecto "**iBichos**" utiliza Firebase Firestore, una base de datos distribuida NoSQL. A diferencia de los motores SQL tradicionales que enlazan intrincadas tablas algebraicas, Firestore preserva la información bajo un formato atómico y plano de **Colecciones y Documentos**, ideal para latencias sub-milisegundo en aplicaciones móviles.

1. Defensa Académica de Arquitectura: Esquemas MongoDB vs Firebase

🤔 Posible Pregunta del Jurado/Profesor:

"Veo que están usando una base de datos NoSQL. En clases pasadas con MongoDB utilizábamos sentencias pesadas como `$jsonSchema`, `bsonType` y `validationAction: "error"` en el backend al crear colecciones para obligar a que los datos no entraran equivocados (por ejemplo, que no metieran un string donde va un número). Aquí en Firebase no ejecutaron ningún script de BD. ¿Están trabajando sin esquemas? ¿Sus datos quedan vulnerables a tipos corruptos?"

✅ Respuesta Técnica:

Los datos están 100% protegidos, estructurados y tipados; la diferencia radica en el paradigma de validación. En vez de usar validación del lado del servidor (**Server-Side Validation**) como se hacía en MongoDB estricto, nosotros usamos **Validación Fuerte del Lado del Cliente (Client-Side Validation)** impulsada por las arquitecturas modernas de Kotlin y Android.

1. **Optimización "Schema-less" en la Nube:** El núcleo Firestore de Google es nativamente *Schema-less*. Su objetivo de mercado es tragar y escupir lectura de datos de manera

atómica instantánea. Si malgastara ciclo de procesador chequeando cada propiedad (como un parser relacional), perdería su legendaria velocidad de red. Por ende, la nube descansa su seguridad en nuestra App en sí misma.

2. **El Escudo Rígido (Data Classes):** Nuestro código Android usa inyecciones fuertemente tipadas obligatorias (`Data Classes`). Son literalmente el equivalente uno-a-uno a un mega script `$jsonSchema` en MongoDB:

```
data class CaptureItem(
    val id: String = "",           // Equivalente a bsonType: "string"
    val probability: Double = 0.0, // Equivalente a bsonType: "double"
    val xpAwarded: Int = 0         // Equivalente a bsonType: "int"
)
```

3. **Física de Compilación Cero:** Si un estudiante de nuestro propio equipo, o una falla en el código intenta inyectar el texto `"mil puntos"` dentro de la variable `xpAwarded`, la **Virtual Machine** de Android se quiebra de inmediato (Compiling Error TypeMismatch) haciendo matemáticamente imposible que un tipo corrupto escale al protocolo BSON y vuele hacia internet.
4. **Resumen de Rendimiento:** *Kotlin serializa hacia JSON.* Toda la purificación de entidades se la cobramos y castigamos al poderoso procesador del Smartphone del estudiante, ****salvando millones de operaciones gratuitas de chequeo en nuestro Servidor Gratuito (MVP)**.**

2. Ausencia de Tablas Redundantes (El Caso del Ranking)

En el diseño canónico de iBichos **no existe una colección 'Top10' o 'Ranking Global' insertada artificialmente.** Se descartó una colección redundante por las siguientes fallas lógicas subyacentes:

- **Dualidad de Estado:** Los Puntos de Experiencia (XP) son dueños intrínsecos del Perfil del Cazador (`users`). Desdoblar este dato hacia una colección extra causaría la infame *violación de la única fuente de verdad*. Si hay un parpadeo de Wi-Fi, una inserción fallaría provocando que un usuario reciba sus XP pero la tabla de ranking no lo refleje.

- **Fuerza Bruta de Indexación:** Al usar *Flat Data*, le ordenamos a Firebase precompilar índices compuestos en el fondo. El Sistema Android solo arroja la red a la colección `Users` y exige: *"Filtra los top 50, en Base Descendiente (Query.Direction.DESENDING) desde el nodo xp"*, resolviendo el desafío sin despilfarro estructural de tablas.

3. Estructura Jerárquica BSON (Diccionario de Datos)

El motor lógico se divide en exactly dos Colecciones Raíces de tipo No-Relacional (Relación mediante Foránea Virtual).

Colección A: `users`

Guarda el progreso de los cazadores y sus coleccionables bajo un identificador criptográfico privado de autenticación.

- **Document ID: [UUID Autenticación]** (Ej: ZpLQ9WbY1...)
 - **email** (*String*): Correo identitario del jugador.
 - **displayName** (*String*): Pantalla Display de dominio público intra-app.
 - **region** (*String*): Región de residencia para perfilamiento.
 - **comuna** (*String*): Comuna asociada a la región elegida.
 - **birthDate** (*String*): Fecha de nacimiento bajo el formato `dd/MM/yyyy`.
 - **gender** (*String*): Sexo o género designado en el formulario.
 - **xp** (*Long*): Acumulado total de experiencia competitiva.
 - **level** (*String*): Calificación de prestigio o rango temporal.
 - **avatarUrl** (*String*): Ruta local o remota a la fotografía de perfil.
 - **totalCaptures** (*Int*): Cantidad total del inventario bruto sin distinción de especies.
 - **uniqueInsectsCount** (*Int*): Diversidad taxonómica total lograda.
 - **medals** (*Array de Strings*): **[DATO EMBEBIDO]** Archiva condecoraciones obtenidas y descarta construir una tabla relacional externa pesada.
 - **categoryCounts** (*Map / JSON Anidado*): **[DATO EMBEBIDO]** Hashmap estadístico que contabiliza familias `{"Arácnidos": 4}` ahorrando procesamientos extra.

Colección B: `captures`

Inventario cronológico infinito y registro biológico AI de todas las exploraciones realizadas globalmente.

- **Document ID: [AlphaNum Aleatorio Firebase]** (Ej: IP8xjTOW2...)
 - **userId** (*String - Foránea Lógica*): Conexión rastreable hacia **users**.
 - **imageUrl** (*String*): Enrutamiento de la fotografía biométrica local/remota.
 - **scientificName / insectName** (*Strings*): Nomenclaturas taxonómicas y populares.
 - **dangerLevel** (*String*): Clasificación sobre la interacción de vida silvestre.
 - **probability** (*Double*): Confianza analítica en [0.0 - 1.0].
 - **latitude / longitude** (*Double*): Vectores satelitales que localizan el área del avistamiento natural.
 - **xpAwarded** (*Long*): Valor estandarizado asignado y sumado durante el dictamen IA.
 - **capturedAt** (*Long UNIX*): Base *Epoch Time* atemporal para esquivar la disparidad de husos horarios inter-dispositivos.
 - **description** (*String*): Fragmento enciclopédico derivado del análisis.